



Rapport du mini-projet LU3IN025

---

## **Affectation des étudiants en Master Informatique**

---

Helder Brito (21304177) et O'nel Hounnouvi (21315612)

Avril 2026 – Mai 2026

# 1 Problème et affectation

## 1.1 Gale-Shapley : Côté étudiants

### 1.1.1 Choix des structures de données pour l'implémentation

- **Étudiants libres (etu\_libres) : File (Queue)**

Implémentée via une deque (Double-Ended Queue), elle permet de stocker les étudiants n'ayant pas encore été affectés à un master.

- *Extraction du premier arrivé* :  $\mathcal{O}(1)$
- *Ajout d'un étudiant rejeté* :  $\mathcal{O}(1)$

- **Progression des propositions (prochain\_voeu) : Tableau**

Au lieu de faire un pop sur la liste de préférences d'un étudiant (et donc la détruire), ce tableau stocke simplement l'indice de sa prochaine proposition. Ainsi, `prochain_voeu[i]` indique l'indice du prochain master à proposer pour l'étudiant  $e_i$  et est incrémenté à chaque proposition. L'instruction `PrefEtu[i][prochain_voeu[i]]` permet d'obtenir le master ciblé sans modifier la liste de préférences.

- *Lecture et Incrémentation* :  $\mathcal{O}(1)$

- **Position de l'étudiant i dans un parcours j (classement) : Matrice**

Une matrice `classement` calculée en amont en inversant la matrice de préférences des masters (`PrefSpe`). Ainsi, `classement[i][j]` stocke le rang de l'étudiant  $i$  pour le master  $j$ . Cela évite au master de parcourir sa liste de préférences en  $\mathcal{O}(n)$  à chaque proposition.

- *Recherche du rang de l'étudiant* :  $\mathcal{O}(1)$

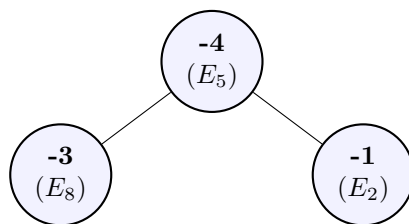
- **Gestion des affectations (AFFECTATIONS) : Tas Min (Min-Heap)**

Pour chaque master, la liste des étudiants acceptés est maintenue dans une file de priorité (Tas binaire). En insérant l'opposé du rang, le pire étudiant (le plus grand rang, donc le plus petit nombre négatif) est toujours à la racine du tas. Si  $c_j$  est la capacité du master  $j$ , le tas permet de:

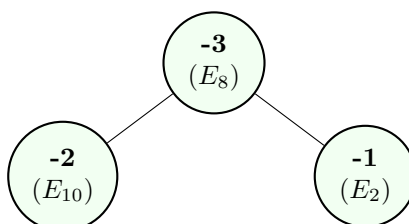
- *Identifier le pire étudiant (Racine)* :  $\mathcal{O}(1)$
- *Insérer un nouvel étudiant* :  $\mathcal{O}(\log c_j)$
- *Remplacer le pire étudiant* :  $\mathcal{O}(\log c_j)$

#### Fonctionnement du Tas Min (Exemple)

Imaginons un parcours de capacité  $c = 3$ . Il a actuellement accepté les étudiants  $E_2$  (rang 1),  $E_8$  (rang 3) et  $E_5$  (rang 4). La racine contient le pire candidat à éjecter en cas de meilleure proposition.



Si un étudiant  $E_{10}$  de rang 2 postule, le master regarde la racine (-4). Puisque  $-2 > -4$ ,  $E_{10}$  remplace  $E_5$ , et le tas se rééquilibre automatiquement en  $\mathcal{O}(\log c)$ .



### 1.1.2 Complexité de l'algorithme

L'algorithme de Gale-Shapley garantit que chaque étudiant propose à chaque master au plus une fois. Avec  $n$  étudiants et  $m$  masters, le nombre total de propositions est au maximum  $n \times m$ .

- *Boucle principale (Propositions)* :  $\mathcal{O}(n \times m)$  dans le pire des cas.
- *Gestion des affectations* : À chaque proposition, l'étudiant est évalué par le master. Dans le pire des cas, cette opération coûte  $\mathcal{O}(\log c_j)$ .

Ainsi, la complexité temporelle pire cas de l'algorithme est de  $\mathcal{O}(n \times m \log c_{\max})$ , où  $c_{\max}$  est la plus grande capacité parmi tous les masters.

**Remarque :** En pratique, les capacités  $c_j$  des masters sont des valeurs petites et fixes, indépendantes du nombre total de candidats  $n$ . Le terme  $\log c_{\max}$  agit donc comme une constante  $\mathcal{O}(1)$  et la complexité effective est  $\mathcal{O}(n \times m)$ .

## 1.2 Gale-Shapley : Côté parcours

### 1.3 Affectations obtenues sur les fichiers tests et stabilité

Table 1: Côté étudiants et côté parcours

| Parcours  | 0             | 1     | 2     | 3     | 4                | 5     | 6     | 7     | 8     | 9          |
|-----------|---------------|-------|-------|-------|------------------|-------|-------|-------|-------|------------|
| Étudiants | $E_5, E_{12}$ | $E_4$ | $E_9$ | $E_8$ | $E_{10}, E_{11}$ | $E_0$ | $E_1$ | $E_7$ | $E_6$ | $E_2, E_3$ |

- On obtient les mêmes affectations en appliquant l'algorithme de Gale-Shapley, que ce soit du côté des étudiants ou du côté des parcours, à l'ordre d'affectation près.
- La stabilité de l'affectation est vérifiée par notre fonction `liste_paires_instables` qui retourne une liste vide, indiquant qu'il n'existe aucune paire instable.

## 2 Évolution du temps de calcul

Voici les graphiques obtenus pour les différentes configurations d'affectation (côté étudiants et côté parcours) en fonction du nombre d'étudiants  $n$  et du nombre de parcours  $m$ .

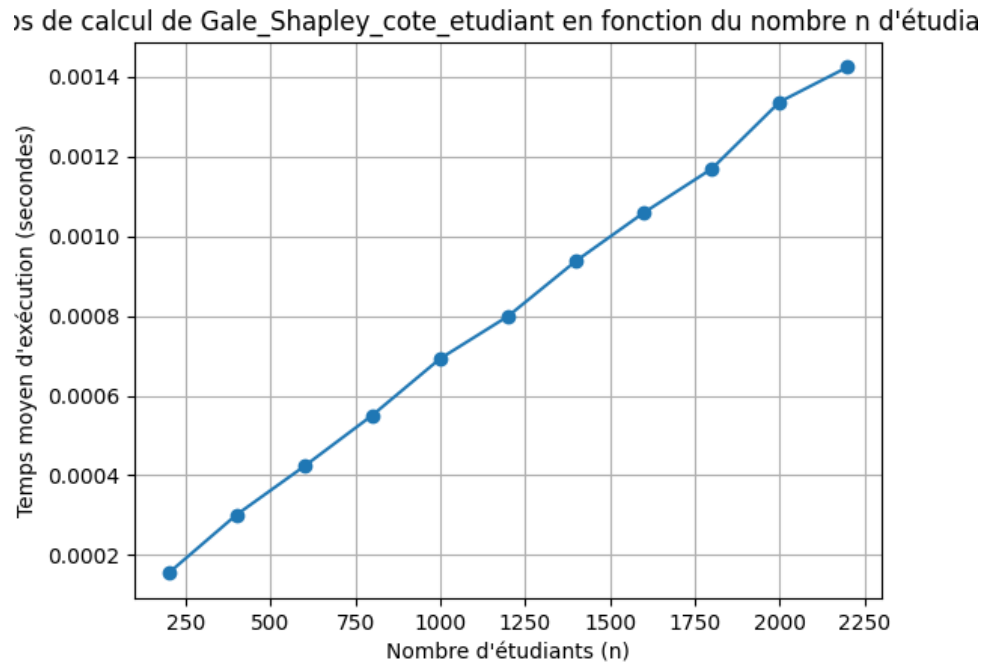


Figure 1: Évolution du temps de calcul en fonction de  $n$  et  $m$

### 3 Équité et PL(NE)

#### 3.1 PLNE permettant de déterminer une affectation qui maximise l'utilité minimale des étudiants.

**Modélisation de la recherche de l'affectation maximisant l'utilité minimale des étudiants**

**Variables:** Pour tout étudiant  $i \in \{0, \dots, 12\}$  et tout parcours  $j \in \{0, \dots, 9\}$ , on définit la variable  $x_{ij}$  telle que  $x_{ij} = 1$  si l'étudiant  $e_i$  est affecté au parcours  $p_j$  et  $x_{ij} = 0$  sinon.

**Fonction objectif :**

$$\max \left( \min_{i \in \{0, \dots, 12\}} \sum_{j=0}^9 x_{ij} u_{e_i}(p_j) \right)$$

La fonction min n'étant pas linéaire, on peut introduire, comme dans le cours, une variable auxiliaire  $Z$  représentant l'utilité du plus malheureux des étudiants pour linéariser le problème. Il se reformule comme suit :

**Fonction objectif :**

$$\max Z$$

**Contraintes :**

$$\begin{aligned} \sum_{j=0}^9 x_{ij} &= 1, & \forall i \in \{0, \dots, 12\} & \quad (\text{chaque étudiant a un seul parcours}) \\ \sum_{i=0}^{12} x_{ij} &\leq c_j, & \forall j \in \{0, \dots, 9\} & \quad (\text{capacité } c_j \text{ respectée}) \\ Z &\leq \sum_{j=0}^9 x_{ij} u_{e_i}(p_j), & \forall i \in \{0, \dots, 12\} & \quad (Z \text{ est borné par l'utilité de chaque étudiant}) \\ x_{ij} &\in \{0, 1\}, \quad Z \geq 0, & \forall i, j & \quad (\text{domaine des variables}) \end{aligned}$$

Table 2: Matrice  $(u_{e_i}(p_j))_{i,j}$  des utilités des étudiants

| Etudiants | Parcours |   |   |   |   |   |   |   |   |   |
|-----------|----------|---|---|---|---|---|---|---|---|---|
|           | 0        | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 0         | 2        | 1 | 3 | 4 | 0 | 9 | 6 | 7 | 5 | 8 |
| 1         | 7        | 0 | 4 | 1 | 6 | 8 | 9 | 5 | 2 | 3 |
| 2         | 8        | 2 | 6 | 3 | 9 | 0 | 1 | 7 | 4 | 5 |
| 3         | 5        | 1 | 0 | 2 | 3 | 7 | 8 | 6 | 4 | 9 |
| 4         | 4        | 9 | 3 | 0 | 2 | 5 | 8 | 6 | 1 | 7 |
| 5         | 9        | 2 | 5 | 3 | 6 | 0 | 1 | 7 | 4 | 8 |
| 6         | 3        | 2 | 6 | 4 | 0 | 9 | 7 | 8 | 5 | 1 |
| 7         | 8        | 2 | 6 | 4 | 7 | 0 | 1 | 9 | 5 | 3 |
| 8         | 3        | 2 | 6 | 4 | 1 | 9 | 7 | 8 | 5 | 0 |
| 9         | 0        | 3 | 9 | 4 | 2 | 7 | 8 | 1 | 5 | 6 |
| 10        | 7        | 4 | 1 | 5 | 8 | 3 | 9 | 0 | 6 | 2 |
| 11        | 0        | 9 | 5 | 4 | 8 | 3 | 2 | 6 | 1 | 7 |
| 12        | 9        | 1 | 3 | 4 | 0 | 2 | 7 | 6 | 5 | 8 |

### 3.2 PLNE permettant de trouver une solution qui maximise la somme des utilités (étudiants et parcours)

#### Modélisation de la recherche de l'affectation maximisant l'utilité totale (efficacité)

Avec les variables  $x_{ij}$  définies précédemment, on peut formuler le problème de la manière suivante :

**Fonction objectif :**

$$\max \sum_{i=0}^{12} \sum_{j=0}^9 x_{ij} (u_{e_i}(p_j) + u_{p_j}(e_i))$$

**Contraintes:**

$$\begin{aligned} \sum_{j=0}^9 x_{ij} &= 1, & \forall i \in \{0, \dots, 12\} & \quad (\text{chaque étudiant est affecté à un seul parcours}) \\ \sum_{i=0}^{12} x_{ij} &\leq c_j, & \forall j \in \{0, \dots, 9\} & \quad (\text{la capacité } c_j \text{ de chaque parcours est respectée}) \\ x_{ij} &\in \{0, 1\}, & \forall i, j & \quad (\text{domaine des variables}) \end{aligned}$$

Table 3: Matrice  $(U_{ij})_{i,j}$  des utilités totales  $u_{e_i}(p_j) + u_{p_j}(e_i)$

| Etudiants | Parcours |    |    |    |    |    |    |    |    |    |
|-----------|----------|----|----|----|----|----|----|----|----|----|
|           | 0        | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
| 0         | 6        | 5  | 6  | 9  | 10 | 20 | 18 | 10 | 16 | 12 |
| 1         | 12       | 6  | 8  | 7  | 7  | 20 | 19 | 9  | 14 | 8  |
| 2         | 8        | 2  | 6  | 3  | 9  | 5  | 4  | 7  | 9  | 5  |
| 3         | 12       | 8  | 12 | 9  | 14 | 17 | 17 | 11 | 14 | 16 |
| 4         | 12       | 17 | 10 | 8  | 9  | 14 | 15 | 13 | 10 | 8  |
| 5         | 18       | 12 | 13 | 12 | 12 | 8  | 7  | 15 | 12 | 17 |
| 6         | 5        | 3  | 11 | 8  | 5  | 16 | 12 | 19 | 12 | 3  |
| 7         | 20       | 13 | 12 | 15 | 11 | 6  | 5  | 21 | 11 | 14 |
| 8         | 4        | 4  | 7  | 5  | 4  | 9  | 9  | 9  | 6  | 8  |
| 9         | 11       | 12 | 19 | 14 | 4  | 10 | 8  | 10 | 9  | 18 |
| 10        | 10       | 7  | 3  | 8  | 20 | 5  | 10 | 2  | 8  | 5  |
| 11        | 10       | 14 | 16 | 16 | 16 | 7  | 13 | 12 | 1  | 17 |
| 12        | 15       | 13 | 12 | 6  | 9  | 3  | 15 | 16 | 8  | 14 |

### 3.3 PLNE maximisant la somme des utilités (étudiants et parcours) parmi les affectations où chaque étudiant a un de ses k premiers choix (pour un k fixé)